

Agent Infrastructure — Team Training Guide

SECTION 06-operations-it

TYPE guide

STATUS **active**

OWNER Founder / Operations

UPDATED 2026-08-26

Version: 1.0 | Effective: 2026-08-26

Purpose. This guide trains internal team members on the Fly GACA agent infrastructure — how to discover agents, trigger workflows, and understand shared constraints across the iflygaca/Office, iflygaca/FlyGACA, and iflygaca/Captain-Adel family. It is **required reading for all new hires in their first week** and **reference material for anyone working with Claude Code in any repo**. See integration into the general onboarding checklist below.

1. What the Agent Layer Is — and Isn't

The Fly GACA family operates as **three synchronized repositories** (Office, FlyGACA, Captain-Adel) with a **bounded internal agent workforce** that **drafts, scaffolds, and automates** — but **does not replace accountability**.

What agents do	What agents do NOT do
Research, write drafts, edit documents	Own business decisions or accountability
Run compliance audits, surface risks	Make strategic calls or override governance
Trigger workflows, coordinate cross-repo tasks	Replace human judgment in sensitive areas
Encode non-inferable repo/domain knowledge	Substitute for subject-matter experts
Automate repetitive, high-fidelity work	Perform creative work beyond their slot

21 agents work across the family, organized into five groups:

- **7 Office agents** — governance, documents, regulatory, entity facts, cross-repo coordination
- **5 Product agents** — React, Express, GACAR corpus, SQL, RAG/Gemini
- **4 Flight agents** — curriculum, ML tuning, learner data, deployment
- **3 GTM agents** — go-to-market coordination, competitive defense, schools acquisition
- **2 Infrastructure agents** — orchestration and cross-repo workflow (shared by all)

An agent is justified only when it **encodes non-inferable, repo-specific knowledge** — facts a generic Claude model cannot infer. The "earn its slot" test is documented in `06-operations-it/agent-workforce-plan.md` (section 2).

2. Agent Roster: Quick Reference

Office Agents (Governance Layer)

These seven agents live in `iflygaca/Office/.claude/agents/` and are available to all three repos via the family contract.

Agent	Scope	Best for	Non-inferable fact
doc-smith	Documents, print pipeline	Creating/editing <code>.md</code> or <code>.html</code> files, rebuilding PDFs when CI fails	The 4-key front-matter schema; <code>build.mjs</code> → <code>themeHash</code> staleness; Falcon Theme styling
ar-mirror	Arabic translation, bilingual mirroring	Translating new/changed English docs to <code>ar/</code> , maintaining EN/AR parity	GLOSSARY.md terminology; Saudi MSA conventions; PDF rebuild pathway
ksa-compliance	PDPL, ZATCA, MISA, Nitaqat	Drafting/reviewing anything turning on Saudi regulation; vetting vendored-skill output	Six Saudi regimes (PDPL, ZATCA, MISA, Monshaat, Nitaqat, ISMS); the <code>04-compliance-ksa/</code> section ownership
family-warden	Family contract, entity facts, cross-repo parity	Changing the contract, checking entity facts against <code>company-facts.md</code> , cross-repo drift sweeps	Byte-identical contract semantics; IBAN/account protection rule; contract stamping procedure
governance-auditor	Decision log, policy drift	Keeping the decision log consistent, tracking policy changes	Decision log schema (DEC-NNN); reversibility markers; stakeholder-field meanings
entity-facts-guardian	Entity facts parity, MCP validation	Ensuring <code>company-facts.md</code> and the family contract stay synchronized	12 entity fields; company facts table semantics; MCP assertion logic
cross-repo-sync	Multi-repo coordination, contract distribution	Synchronizing PRs across Office, FlyGACA, and Captain-Adel when contract or entity facts change	Three-repo stamping workflow; PR title/body conventions; simultaneous-merge timing

Product Agents (iflygaca/FlyGACA)

These five agents are documented in `iflygaca/FLyGACA/CLAUDE.md` and handle the web app and backend.

Agent	Scope	Best for
react-19-architect	React 19, Vite, TypeScript strict, RTL/i18n	Component design, strict TS patterns, RTL logical properties, i18n hooks
express-backend-pro	Express 5, Cloud Run, API security	API endpoints, parameterized queries, HttpOnly JWT, server-owned entitlements
regulatory-corpus-keeper	GACAR indexing, corpus tiers, AIRAC freshness	Citation verification, corpus policy enforcement, stale-content detection

Agent	Scope	Best for
sql-migrator	PostgreSQL, forward-only migrations, indexes	Schema design, migration safety, query optimization, index strategy
genkit-rag-specialist	Gemini integration, RAG pipeline, grounding	Prompt design, RAG chunking, inference safety, PDPL-compliant grounding

Flight Agents (iflygaca/Captain-Adel)

These four agents are documented in `iflygaca/Captain-Adel/CLAUDE.md` and handle the flight instructor service.

Agent	Scope	Best for
flight-curriculum-designer	Syllabus, mock exams, learner paths, safety content	Pedagogical design, mastery gates, GACAR alignment, safety-critical markers
ml-instructor-trainer	Model fine-tuning, persona, eval metrics, confusion detection	Instructor persona tuning, eval harness, confusion signals, knowledge retention
flight-data-pipeline-engineer	Learner data, flight-hour tracking, currency, PDPL	Data schema design, currency calculations, PDPL-compliant ingestion
instructor-deployment-steward	Cloud Run deployment, captadel.com, versioning	Revision management, webhook routing, version pinning, rollback procedures

GTM Agents (iflygaca/Office)

These three agents coordinate go-to-market work across the family.

Agent	Scope	Best for
fly-gaca-gtm-orchestrator	Cross-repo GTM, schools motion, pricing, competition	Schools sales, pricing alignment, competitive threat analysis, ARR composition
gtm-defensibility-steward	Brand positioning, defensible wedges	Competitive positioning, brand constraints, NTSB↔GACAR differentiation
schools-acquisition	B2B schools outreach, pilot playbook	Schools pilot program, conversion discipline, 14-day playbook execution

Infrastructure Agents (Shared)

Agent	Scope	Role
operations-orchestrator	Multi-agent workflow coordination	Conductor of the five cross-repo workflows; gates between agents

3. How to Discover and Use Agents

Step 1: List agents in any repo

```
ls .claude/agents/
```

This shows all agents available to that repo. In `iflygaca/Office`, you see all 7 Office + shared infrastructure agents. In `iflygaca/FLYGACA`, you see the 5 product agents + Office agents + shared. In `iflygaca/Captain-Adel`, you see the 4 flight agents + Office agents + shared.

Step 2: Read an agent's documentation

Each `.md` file under `.claude/agents/` carries:

- **name** — the kebab-case agent ID (e.g., `doc-smith`)
- **description** — one-line purpose
- **tools** — what it can do (Read, Write, Edit, Bash, etc.)
- **color** — visual identifier in Claude Code UI
- **Second-person imperative prose** — what non-inferable knowledge it encodes and how to invoke it
- **Report contract** — what output you get

Example: To use `doc-smith` for editing a document:

1. You edit a `.md` or `.html` file
2. If `npm run check` CI fails on front-matter or PDF freshness
3. Ask doc-smith to fix it — it knows the front-matter schema, the PDF rebuild pathway, and the Falcon Theme styling

Step 3: Call an agent in Claude Code

In **Claude Code**, type:

```
@doc-smith [task description]
```

Or use the agent selector in the sidebar. The agent runs with pre-approved tools and repo context.

4. Shared Constraints: Every Agent Must Know These

These constraints are **non-negotiable across all three repos**. They are not optional; they are load-bearing.

Data Residency & PDPL

- **Only** `me-central2` **Dammam** — never `me-central1` (Doha, Qatar — not PDPL-safe)
- **Learner data minimal:** Name, email, progress only. No passport, address, biometrics, voice recordings, face data.
- **Flight-hour log:** Timestamp, duration, aircraft type, instructor name, exam module — no route details, no ADS-B data.
- **Gemini inference outside Kingdom** — open risk, documented, not hidden.
- **IBAN and account number never in the family contract** — they live in `01-governance/company-facts.md` only.

Regulatory Authority

- **GACAR is authoritative** — Every citation verified against official GACAR sections; never fabricate.
- **Three corpus tiers:**
 - **HOST_SAFE_CORE:** Can appear on learning interface and exam (vetted, approved)
 - **HOST_ORIGINAL:** Can appear in study materials but not public (proprietary)
 - **DO_NOT_HOST:** Cite only (external links, reference books)
- **AIRAC freshness:** Content is stale 28 days after effective date; 7-day buffer for updates (35-day threshold)
- **GACA non-affiliation** — Fly GACA is **not** affiliated with GACA; all materials state the relationship accurately.

API & Backend Security

- **Parameterized queries always** — no SQL concatenation.
- **HttpOnly JWT tokens** — never in localStorage, URLs, or bearer tokens in frontend; always secure flag.
- **Server-owned entitlements** — Backend verifies access; frontend never trusts roles.
- **No** `dangerouslySetInnerHTML` — All user input sanitized.
- **CORS whitelist specific** — never `*`.
- **Rate limiting on auth endpoints** — brute-force protection (10 failed attempts per 15 min per IP).

Flight-Hour Schema

- **No external flight-ops APIs** — Do not pull from flight-tracking APIs, ADS-B, or airline crew systems.
- **No voice/audio** — Instructor-learner conversations never recorded; interaction logged as text-only event markers.
- **Currency calculation forward-looking** — Rolling 12-month total per exam module; regulatory compliance per module.

React & Frontend (FlyGACA)

- **TypeScript strict** — No `any` types.
- **RTL properties** — `margin-inline`, `padding-inline`, not `margin-left/right`.
- **i18n hooks** — All user-facing strings via `i18n`, never hardcoded text.
- **CSS Modules** — No inline styles; design tokens via CSS custom properties.

Instructor Persona (Captain-Adel)

- **Warm, challenging, culturally aware** — Encourages mastery; does not patronize; understands prayer times, holidays, values.
- **Confusion detection** — Tracks conceptual gaps; re-teaches via analogy or first principles.
- **Progression gating** — Learner cannot advance without demonstrating mastery ($\geq 80\%$ on skill-checks).
- **Safety-critical content** — Marked explicitly; never simplified or speculated; backed by SOP references.

5. The Five Workflows: What They Do and When to Use Them

Five cross-repo workflows are triggered via **slash commands** in Claude Code or via **scheduled automation** (Sundays 18:00 UTC for `/full-sync`). Each workflow is an **orchestration** — it chains multiple agents together to accomplish a multi-repo task.

`/full-sync` — Weekly Family Health Check

When: Automatically Sunday 18:00 UTC, or manually anytime.

Agents: entity-facts-guardian, cross-repo-sync, governance-auditor.

Duration: ~15 minutes.

What it does:

1. Verifies entity facts in `01-governance/company-facts.md` match the family contract.
2. Checks the family contract SHA256 is byte-identical across all three repos.
3. Audits the decision log for drift, orphaned entries, or inconsistent reversibility markers.
4. Reports any mismatches.

Output: Summary report of Office ↔ FlyGACA ↔ Captain-Adel parity. Green = all aligned; red = drift detected.

`/feature-ship <name>` — Coordinate Feature Across All Repos

When: Before shipping a feature that touches multiple repos (React UI + Express API + curriculum, for example).

Agents: react-19-architect + express-backend-pro + ksa-compliance + cross-repo-sync.

Duration: ~30 minutes.

What it does:

1. react-19-architect reviews React component changes.
2. express-backend-pro reviews API changes and security.
3. ksa-compliance gates any regulatory impact.
4. cross-repo-sync coordinates the synchronized PR merge.

Output: Coordinated merge; all three PRs land together.

`/compliance-audit` — Quarterly Compliance Review

When: Quarterly, or before investor calls.

Agents: ksa-compliance, flight-data-pipeline-engineer.

Duration: ~45 minutes.

What it does:

1. Audits PDPL boundary — learner data schema, what is logged, where it is stored.
2. Checks ZATCA readiness, Nitaqat status, MISA compliance.
3. Reviews flight-hour schema for privacy leaks.
4. Checks breach procedures are current.

Output: Compliance posture report; any gaps flagged for remediation.

`/security-hardening` — Security Review Across Stack

When: Before public releases, after threat alerts, or quarterly.

Agents: react-19-architect + express-backend-pro + ksa-compliance.

Duration: ~60 minutes.

What it does:

1. react-19-architect audits for XSS, dependency vulnerabilities, localStorage misuse.
2. express-backend-pro audits for injection, broken authentication, misconfiguration.
3. ksa-compliance checks data-residency and PDPL boundary compliance.

Output: Security audit report; prioritized fix list.

`/perf-sprint` — Performance Optimization

When: When latency or throughput issues surface.

Agents: react-19-architect + express-backend-pro + sql-migrator.

Duration: ~60 minutes.

What it does:

1. react-19-architect profiles bundle size, renders, i18n overhead.
2. express-backend-pro profiles endpoint latency, concurrency limits, error rates.
3. sql-migrator analyzes query performance, index usage, data-loading patterns.

Output: Performance diagnostic; optimizations ranked by impact/effort.

6. First Hands-On: Triggering `/full-sync`

Your first hands-on workflow is `/full-sync` — it runs weekly automatically and can be triggered manually anytime to verify the family is healthy.

Before You Start

- You are in `iflygaca/Office` (any branch).
- You have read this guide through section 5.

The Walkthrough

1. Open Claude Code in `iflygaca/Office`.

```
cd ~/Office  
claude code .
```

2. In the Claude Code chat, type:

```
/full-sync
```

3. Claude Code loads the **full-sync** workflow and launches the three agents:

- entity-facts-guardian checks `01-governance/company-facts.md` vs. the family contract.
- cross-repo-sync verifies the contract SHA256 across all three repos.
- governance-auditor scans the decision log for consistency.

4. You see a report. It will say either:

- ✓ **All aligned** — entity facts, contract SHA, decision log are in sync. Your work is ready.
- ⚠ **Drift detected** — e.g., `company-facts.md` mentions a repo that's not in the family contract, or the decision log has an orphaned entry. The agents flag what changed and what needs fixing.

5. If there's drift, the agents recommend fixes. You review and apply them.

6. You commit the changes (if any) and push. The `/full-sync` workflow is complete.

What to Expect

On your first run, `/full-sync` should report ✓ **All aligned** because Phase 3 deployment just completed and everything was synchronized. If you see any drift, it means:

- Someone edited `company-facts.md` but didn't update the family contract.
- Someone created a decision-log entry but didn't fill in all the fields.
- Someone changed a repo reference but didn't update the manifest.

These are **rare, high-stakes issues** — the workflow catches them before they silently propagate across three repos.

7. Integration with New-Hire Onboarding

This guide is integrated into the general new-hire onboarding checklist (`05-people/onboarding-checklist-2026-07-03.md`). New hires follow this training in **Week 1, Day 3 (after tool orientation and code-base intro)**.

Addition to Onboarding Checklist

Add this line to **Phase C — Week 1 (Days 2–7)**, after the "Codebase / product orientation" line:

- Agent infrastructure orientation: Read this guide (`06-operations-it/agent-infrastructure-team-training.md`); trigger `/full-sync` manually to understand workflow automation – engineering/operations roles

Expectations by Role

Role	Minimum competency
Engineering (product/flight)	Understand agent roster; can discover agents (<code>ls .claude/agents/</code>); knows the five shared constraints (me-central2, PDPL, GACAR, HttpOnly JWT, flight-hour schema); has triggered <code>/full-sync</code> once
Operations	Same as engineering + can interpret <code>/full-sync</code> reports and <code>/compliance-audit</code> output
Compliance	Same + deep familiarity with PDPL, ZATCA, and Nitaqat sections of this guide
GTM/Sales	High-level understanding of agent infrastructure as a differentiation story; knows the three GTM agents exist and what they cover
Customer Success	Knows agents power the family workflows; can escalate to ops if a <code>/compliance-audit</code> flag arises

8. Learning Resources

Resource	Purpose	Who uses it
This guide	Agent overview, roster, constraints, workflows, first hands-on	All new hires, week 1
<code>06-operations-it/agent-workforce-plan.md</code>	Why the roster is bounded, the "earn its slot" test, target rosters for each repo	Ops lead, agent oversight
<code>iflygaca/Office/.claude/agents/README.md</code>	Detailed agent table, shared constraints, retirement notices	Anyone invoking agents
<code>iflygaca/FlyGACA/CLAUDE.md</code>	Product agent reference (5 agents, 3 workflows scoped to product)	Product engineers

Resource	Purpose	Who uses it
iflygaca/Captain-Adel/CLAUDE.md	Flight agent reference (4 agents, 3 workflows scoped to flight service)	Flight engineers
01-governance/decision-log.md	Corporate decisions, reversibility, stakeholders, review dates	Ops lead, governance auditor
contracts/flygaca-family.json	The shared manifest (entity facts, chat contract, repo roster)	cross-repo-sync, family-warden agents

9. Troubleshooting & FAQ

Q: I triggered `/full-sync` and it says the contract SHA doesn't match. What do I do?

A: The family contract is out of sync across the three repos. This is **rare and high-priority**. Contact the ops lead immediately. Do NOT edit the contract manually — use `cross-repo-sync` to re-stamp and distribute the fix.

Q: I'm editing a document and the PDF didn't rebuild. CI is failing.

A: You forgot to rebuild the PDF after editing the `.md`. Run:

```
cd tools/print
npm run build
```

Then commit both the `.md` and the regenerated PDF together. The `doc-smith` agent can also run this if you ask.

Q: An agent I invoked doesn't have permission to do something. What now?

A: Agents have tool access pre-approved at agent-definition time (see `.claude/agents/*.md`). If an agent lacks a tool it needs:

1. Check the tool list in the agent's `.md` file.
2. If the tool is missing, it's by design (e.g., `doc-smith` cannot push to GitHub — that keeps document edits separate from infrastructure changes).
3. If you genuinely need the tool, escalate to the ops lead for a design review.

Q: Can I invoke an agent from a repo where it's not "scoped" to work?

A: No. Agents are only available in repos where they are documented in `CLAUDE.md`. For example, `react-19-architect` is available in `iflygaca/FLyGACA` only. If you need React guidance in another repo, ask Claude directly — it won't have the pre-approved tools, but you can still get advice.

Q: How often should I run `/full-sync`?

A: Automatically, every Sunday 18:00 UTC via the supervisor. Manually, whenever you:

- Commit a cross-repo change (touching `company-facts.md`, `decision-log.md`, or the family contract).
- Merge a feature that touched multiple repos.
- Before investor or compliance calls.

10. What's Next?

After completing this training, you are ready to:

1. **Invoke agents** in your daily work (ask `@doc-smith` to fix a failing PDF, `@ksa-compliance` to vet a new compliance doc, etc.).
2. **Trigger workflows** when your work spans multiple repos (`/feature-ship`, `/compliance-audit`).
3. **Contribute to agent oversight** — report drift, suggest improvements to the roster, propose new agents (they earn their slot, so make the case).
4. **Participate in the weekly** `/full-sync` **review** (Sundays 18:00 UTC report is posted to Slack).

11. Quick-Reference Cheat Sheet

Task	Command	Agent
List available agents	<code>ls .claude/agents/</code>	—
Fix a failing PDF build	<code>@doc-smith</code>	doc-smith
Verify compliance of new doc	<code>@ksa-compliance</code>	ksa-compliance
Trigger weekly health check	<code>/full-sync</code>	operations-orchestrator
Ship a cross-repo feature	<code>/feature-ship <name></code>	react-19-architect + express-backend-pro
Run quarterly compliance audit	<code>/compliance-audit</code>	ksa-compliance + flight-data-pipeline-engineer
Check security posture	<code>/security-hardening</code>	react-19-architect + express-backend-pro
Optimize performance	<code>/perf-sprint</code>	react-19-architect + express-backend-pro + sql-migrator
Update entity facts	<code>@entity-facts-guardian</code> + <code>@family-warden</code>	family-warden (gates the contract)
Translate a new English doc to Arabic	<code>@ar-mirror</code>	ar-mirror

End of Guide. Questions? Escalate to the ops lead or reply in Slack: `#fly-gaca-team`.

Fly GACA / Agent Infrastructure Training / Confidential — Internal